PROJECT DOCUMENTATION

Online Fraud Transaction Detection

scikit-learn · SMOTE · MLflow · FastAPI · GitHub Actions

Abstract

This document describes an end-to-end machine learning system that classifies online payment transactions as fraudulent or legitimate. Three classifiers — Logistic Regression, Decision Tree and Random Forest — are trained inside imbalanced-learn pipelines that combine feature scaling, one-hot encoding and SMOTE oversampling. Every run is tracked in MLflow, the model with the best precision-recall AUC is persisted, and a FastAPI service exposes it through a REST `/predict` endpoint. A GitHub Actions workflow runs the API test suite on every push and pull request to `main`.

Repository: github.com/nandiniarjun03/Online-Fraud-Detection

Author: nandiniarjun03

October 2026

Contents

1. Introduction 3
2. System Architecture 4
3. Dataset 4
4. Training Pipeline 6
5. Prediction API 10
6. Testing and Continuous Integration 12
7. Installation and Usage 13
8. Design Decisions, Limitations and Future Work 14

1. Introduction

Online payment fraud is rare compared with the volume of legitimate traffic, which makes it a classic **imbalanced classification** problem: a model that always answers “not fraud” can reach very high accuracy while catching nothing. This project addresses that by (a) rebalancing the training data with SMOTE and class weights, and (b) selecting models on **precision-recall AUC**, a metric that stays informative when the positive class is scarce.

1.1 Objectives

- Build a reproducible preprocessing and training pipeline for transaction data.
- Compare several classical classifiers under the same preprocessing and resampling strategy.
- Track parameters, metrics and model artifacts for every experiment with MLflow.
- Serve the best model as a low-latency REST API with request validation.
- Run automated tests in continuous integration on every change.

1.2 Technology stack

Layer	Tools	Role in the project
Data handling	pandas, NumPy	Loading, sampling and cleaning the CSV dataset
Modelling	scikit-learn	Scaling, encoding, train/test split, classifiers, metrics
Imbalance	imbalanced-learn	SMOTE oversampling inside a leakage-safe pipeline
Experiment tracking	MLflow (+ azureml-mlflow)	Logging metrics and models per run; Azure ML compatible
Serving	FastAPI, Pydantic, Uvicorn	REST endpoints, input schema validation, ASGI server
Persistence	joblib	Serialising the best pipeline to models/model.pkl
CI	GitHub Actions, pytest, httpx	Automated API tests on push / pull request

2. System Architecture

The project is split into an offline **training** stage and an online **serving** stage. They share a single artifact: the fitted scikit-learn/imbalanced-learn pipeline saved at `models/model.pkl`. Because preprocessing lives inside that pipeline, the API can pass raw transaction fields straight to the model — there is no separate feature-engineering code to keep in sync.

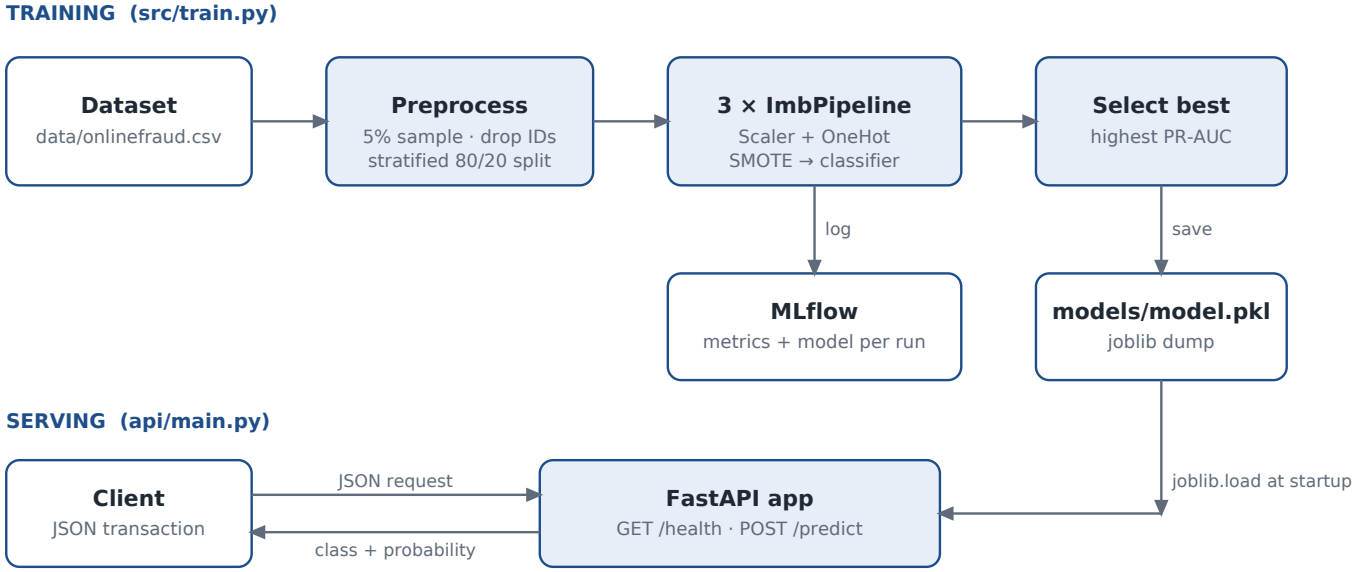


Figure 1 — Training and serving flow.

2.1 Repository layout

Project structure

```
Online-Fraud-Detection/
├── src/
│   └── train.py             # data prep, model comparison, MLflow logging, model export
├── api/
│   ├── main.py              # FastAPI application (/health, /predict)
│   └── test_main.py         # pytest suite using FastAPI's TestClient
├── .github/workflows/
│   └── ci.yml               # GitHub Actions CI pipeline
├── requirements.txt         # pinned Python dependencies
├── .gitignore               # excludes data/, models/, mlruns/, mlflow.db
└── README.md

Generated locally (git-ignored):
data/onlinefraud.csv   models/model.pkl   mlruns/   mlflow.db
```

3. Dataset

Training expects a CSV file at `data/onlinefraud.csv`. The dataset is not committed to the repository and must be obtained separately. Its schema follows the widely used PaySim-style online payments dataset, where each row is one transaction.

Column	Type	Description	Used?
`step`	int	Time step of the simulation (1 step = 1 hour)	Dropped
`type`	category	Transaction type, e.g. CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER	Feature

Column	Type	Description	Used?
`amount`	float	Transaction amount	Feature
`nameOrig`	string	Customer ID that started the transaction	Dropped
`oldbalanceOrg`	float	Sender balance before the transaction	Feature
`newbalanceOrig`	float	Sender balance after the transaction	Feature
`nameDest`	string	Recipient ID	Dropped
`oldbalanceDest`	float	Recipient balance before the transaction	Feature
`newbalanceDest`	float	Recipient balance after the transaction	Feature
`isFraud`	0 / 1	Ground-truth fraud label	Target
`isFlaggedFraud`	0 / 1	Flag raised by a legacy rule-based system	Dropped

Identifier columns (`nameOrig`, `nameDest`) carry no generalisable signal and would let the model memorise accounts; `step` is a simulation clock; and `isFlaggedFraud` is the output of another detector, so using it would leak a second model's decision into this one. The six remaining columns are exactly the fields the API accepts.

4. Training Pipeline

All training logic lives in `src/train.py` in a single function, `train_and_compare_models()`. The script resolves paths relative to the project root, so it can be launched from any working directory.

4.1 Configuration and paths

src/train.py (lines 16–18)

```
16  PROJECT_ROOT = Path(__file__).resolve().parent.parent
17  DATA_PATH = PROJECT_ROOT / "data" / "onlinefraud.csv"
18  MODEL_PATH = PROJECT_ROOT / "models" / "model.pkl"
```

4.2 Loading and cleaning the data

An MLflow experiment named Online_Fraud_Detection is selected (and created on first use). A reproducible **5% random sample** of the dataset is used to keep training fast on a laptop; the unused columns are dropped and rows with missing values are removed. `errors="ignore"` keeps the script working even if a dataset variant omits one of those columns.

src/train.py (lines 20–29)

```
20  def train_and_compare_models(data_path: str | Path = DATA_PATH):
21      mlflow.set_experiment("Online_Fraud_Detection")
22
23      # 5% data for fast execution
24      df = pd.read_csv(data_path).sample(frac=0.05, random_state=42)
25      df.drop(['step', 'nameOrig', 'nameDest', 'isFlaggedFraud'], axis=1, errors='ignore',
            inplace=True)
26      df.dropna(inplace=True)
27
28      X = df.drop('isFraud', axis=1)
29      y = df['isFraud']
```

4.3 Train / test split

The data is split 80/20 with `stratify=y`, so the very small fraud fraction is represented in the same proportion in both sets. Without stratification, a random split of a 5% sample could leave the test set with too few fraud cases to measure recall reliably.

src/train.py (lines 31–33)

```
31      X_train, X_test, y_train, y_test = train_test_split(
32          X, y, test_size=0.2, stratify=y, random_state=42
33      )
```

4.4 Feature preprocessing

A `ColumnTransformer` standardises the five numeric balance/amount columns (zero mean, unit variance) and one-hot encodes the categorical `type` column. `handle_unknown="ignore"` means an unseen transaction type at inference time is encoded as all zeros instead of raising an error.

src/train.py (lines 35–42)

```
35      categorical_features = ['type']
36      numerical_features = ['amount', 'oldbalanceOrg', 'newbalanceOrig', 'oldbalanceDest',
            'newbalanceDest']
37
38      preprocessor = ColumnTransformer(
```

```
src/train.py (lines 35–42)
39          transformers=[
40              ('num', StandardScaler(), numerical_features),
41              ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
42          ])
```

4.5 Candidate models

Three classifiers of increasing capacity are compared. Each uses `class_weight="balanced"`, which weights errors on the minority class more heavily, on top of SMOTE oversampling. `random_state=42` is fixed everywhere for reproducible results.

```
src/train.py (lines 44–49)
44      # Define the 3 models
45      models = {
46          "Logistic_Regression": LogisticRegression(max_iter=1000, class_weight='balanced',
                random_state=42),
47          "Decision_Tree": DecisionTreeClassifier(class_weight='balanced', random_state=42),
48          "Random_Forest": RandomForestClassifier(n_estimators=50, class_weight='balanced',
                random_state=42, n_jobs=-1)
49      }
```

Model	Why it is included
Logistic Regression	Fast, interpretable linear baseline; benefits from the feature scaling step.
Decision Tree	Captures non-linear rules such as “balance drained to zero after a TRANSFER”.
Random Forest	Ensemble of 50 trees; typically the most robust on tabular fraud data. `n_jobs=-1` trains trees in parallel on all CPU cores.

4.6 SMOTE inside an imbalanced-learn pipeline

SMOTE (Synthetic Minority Over-sampling Technique) creates synthetic fraud examples by interpolating between existing fraud cases and their nearest neighbours. The key design choice is that SMOTE is a step inside an `imblearn.pipeline.Pipeline` rather than being applied to the data beforehand:

- Resampling happens only during **fit**. When the pipeline later calls **predict** on the test set or on API requests, the SMOTE step is skipped, so evaluation reflects real class proportions.
- SMOTE runs **after** the split and after scaling, so no synthetic sample is built from test data — this avoids the data leakage that inflates scores when oversampling is done on the full dataset.
- The whole object — preprocessing, resampler and classifier — is saved as one artifact, guaranteeing identical transformations at training and inference time.

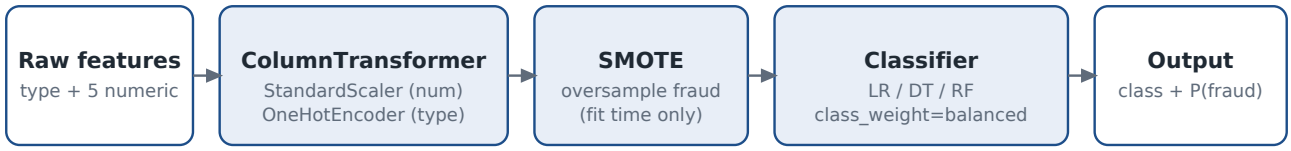


Figure 2 — Steps of each model pipeline.

4.7 Training, evaluation and experiment tracking

Each model is trained inside its own MLflow run. After fitting, the pipeline produces hard predictions (`predict`) and fraud probabilities (`predict_proba`), from which five metrics are computed and logged. The fitted pipeline itself is also logged as an MLflow model, using the cloudpickle format so the custom imbalanced-learn pipeline can be restored later.

src/train.py (lines 54–89)

```
    # Train and log each model automatically
    for model_name, classifier in models.items():
        with mlflow.start_run(run_name=model_name):
            print(f"Training {model_name}...")

            pipeline = ImbPipeline([
                ('preprocessor', preprocessor),
                ('smote', SMOTE(random_state=42)),
                ('classifier', classifier)
            ])

            pipeline.fit(X_train, y_train)
            y_pred = pipeline.predict(X_test)
            y_probs = pipeline.predict_proba(X_test)[:, 1]

            precisions, recalls, _ = precision_recall_curve(y_test, y_probs)
            pr_auc = auc(recalls, precisions)

            precision = precision_score(y_test, y_pred)
            recall = recall_score(y_test, y_pred)
            f1 = f1_score(y_test, y_pred)
            roc_auc = roc_auc_score(y_test, y_probs)

            mlflow.log_metric("Precision", precision)
            mlflow.log_metric("Recall", recall)
            mlflow.log_metric("F1_Score", f1)
            mlflow.log_metric("ROC_AUC", roc_auc)
            mlflow.log_metric("PR_AUC", pr_auc)
            mlflow.sklearn.log_model(
                sk_model=pipeline,
                artifact_path=model_name,
                serialization_format=mlflow.sklearn.SERIALIZATION_FORMAT_CLOUDPICKLE
            )


            print(f"{model_name} PR-AUC: {pr_auc:.4f}\n")
```

4.8 Evaluation metrics

Metric	Meaning in a fraud context
Precision	Of the transactions flagged as fraud, the share that really were fraud. Low precision means many legitimate customers are blocked.
Recall	Of all real frauds, the share the model caught. Low recall means fraud slips through.
F1-score	Harmonic mean of precision and recall at the default 0.5 decision threshold.
ROC-AUC	Threshold-independent ranking quality. Can look optimistic on heavily imbalanced data because true negatives dominate.
PR-AUC	Area under the precision-recall curve. Focuses on the positive (fraud) class and is therefore used as the **model-selection criterion**.

4.9 Model selection and export

The pipeline with the highest PR-AUC is kept in memory during the loop and, once all three runs finish, serialised with joblib to `models/model.pkl`. This is the file the API loads.

src/train.py (lines 91–99)

```
91              # Keep track of the best model for FastAPI
92              if pr_auc > best_pr_auc:
93                  best_pr_auc = pr_auc
94                  best_model = pipeline
95
96      # Overwrite model.pkl with the best performing model (likely Random Forest)
97      MODEL_PATH.parent.mkdir(parents=True, exist_ok=True)
98      joblib.dump(best_model, MODEL_PATH)
99      print(f"Best model saved for API with PR-AUC: {best_pr_auc:.4f}")
```

Results. Concrete metric values depend on the dataset copy and are recorded per run in MLflow. Run `mlflow ui` from the project root (it reads the local tracking store) to compare the three runs side by side, inspect their metrics and download the logged models.

5. Prediction API

`api/main.py` wraps the trained pipeline in a FastAPI application. FastAPI generates interactive OpenAPI documentation automatically at `/docs` (Swagger UI) and `/redoc`.

5.1 Model loading and CI fallback

The model is loaded once, when the module is imported, so individual requests do not pay the deserialisation cost. Because model files are git-ignored, the CI runner has no `model.pkl`; in that case a small `DummyModel` with the same interface is used so the API can still start and its request/response contract can be tested.

api/main.py (lines 7–17)

```
7  app = FastAPI()
8
9  # Check if model exists (for local), else use a Dummy for GitHub Actions (CI)
10 model_path = Path(__file__).resolve().parent.parent / "models" / "model.pkl"
11 if model_path.exists():
12     model = joblib.load(model_path)
13 else:
14     class DummyModel:
15         def predict(self, x): return [0]
16         def predict_proba(self, x): return [[1.0, 0.0]]
17     model = DummyModel()
```

5.2 Request schema

The Pydantic model `TransactionData` defines the expected JSON body. FastAPI validates every request against it and returns HTTP **422** with a descriptive error if a field is missing or has the wrong type — before the model is ever called.

api/main.py (lines 19–25)

```
19 class TransactionData(BaseModel):
20     type: str
21     amount: float
22     oldbalanceOrg: float
23     newbalanceOrig: float
24     oldbalanceDest: float
25     newbalanceDest: float
```

5.3 Endpoints

api/main.py (lines 27–44)

```python
@app.get("/health")
def health_check():
    return {"status": "healthy"}

@app.post("/predict")
def predict_transaction(data: TransactionData):
    try:
        input_data = pd.DataFrame([data.model_dump()])

        prediction = model.predict(input_data)
        probability = model.predict_proba(input_data)[0][1]

        return {
            "predicted_class": int(prediction[0]),
            "fraud_probability": float(probability)
        }
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

Method & path	Description	Response
`GET /health`	Liveness check for load balancers and monitoring.	`{"status": "healthy"}`
`POST /predict`	Scores one transaction. The body is converted to a one-row DataFrame whose column names match the training features.	`predicted_class` (0 = legitimate, 1 = fraud) and `fraud_probability` (0–1)

Any exception raised while scoring is converted into an HTTP 400 response containing the error message, so clients receive a structured error rather than an unhandled 500.

5.4 Example request

Request

```
curl -X POST http://127.0.0.1:8765/predict \
     -H 'Content-Type: application/json' \
     -d '{
           "type": "TRANSFER",
           "amount": 181.0,
           "oldbalanceOrg": 181.0,
           "newbalanceOrig": 0.0,
           "oldbalanceDest": 0.0,
           "newbalanceDest": 0.0
         }'
```

Response (illustrative values)

```
{
  "predicted_class": 1,
  "fraud_probability": 0.97
}
```

The example shows a typical fraud pattern in this kind of data: a **TRANSFER** that empties the sender's account, with no corresponding balance change on the recipient side. The exact probability returned depends on the trained model.

6. Testing and Continuous Integration

6.1 API tests

`api/test_main.py` uses FastAPI's `TestClient` (built on httpx) to call the application in-process, without starting a server. The tests check that the health endpoint responds correctly and that a valid transaction returns both prediction fields.

api/test_main.py (lines 1–23)

```python
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_health_check():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "healthy"}

def test_predict_transaction():
    payload = {
        "type": "PAYMENT",
        "amount": 9839.64,
        "oldbalanceOrg": 170136.0,
        "newbalanceOrig": 160296.36,
        "oldbalanceDest": 0.0,
        "newbalanceDest": 0.0
    }
    response = client.post("/predict", json=payload)
    assert response.status_code == 200
    assert "predicted_class" in response.json()
    assert "fraud_probability" in response.json()
```

6.2 GitHub Actions workflow

The workflow in `.github/workflows/ci.yml` runs on every push and pull request targeting `main`. It provisions Python 3.12 on an Ubuntu runner, installs the pinned requirements plus the test tools, and runs pytest from inside the api directory so the import of main resolves.

.github/workflows/ci.yml (lines 1–26)

```yaml
name: CI Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v3
    - name: Set up Python
      uses: actions/setup-python@v4
      with:
        python-version: '3.12'
    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install -r requirements.txt
        pip install pytest httpx
```

.github/workflows/ci.yml (lines 1–26)

```
      - name: Run tests
        run: |
          cd api
          pytest test_main.py
```

7. Installation and Usage

7.1 Prerequisites

- Python 3.12 (the version used in CI).
- The dataset saved as `data/onlinefraud.csv` in the project root.

7.2 Step by step

Terminal

```
# 1. Install dependencies
python -m pip install -r requirements.txt

# 2. Train, compare and export the best model  ->  models/model.pkl
python src/train.py

# 3. (optional) Browse the experiment runs at http://127.0.0.1:5000
mlflow ui

# 4. Start the API from the project root
python -m uvicorn main:app --app-dir api --host 127.0.0.1 --port 8765

# 5. Run the tests
python -m pip install pytest httpx
python -m pytest api/test_main.py
```

Once the server is running, open `http://127.0.0.1:8765/docs` to try the endpoints from the browser.

7.3 Dependencies

Package	Pinned version
pandas	2.3.3
numpy	2.5.3
scikit-learn	1.9.1
fastapi	0.136.1
uvicorn	0.35.0
pydantic	2.12.5
joblib	1.5.3
imbalanced-learn	0.14.2
mlflow	unpinned (latest)
azure-ai-ml	unpinned (latest)
azureml-mlflow	unpinned (latest)

8. Design Decisions, Limitations and Future Work

8.1 Key design decisions

- **Single pipeline artifact** — preprocessing, resampling and the classifier are versioned and deployed together, eliminating training/serving skew.
- **PR-AUC for selection** — reflects performance on the rare fraud class better than accuracy or ROC-AUC.
- **Double imbalance handling** — SMOTE plus class weights push every model toward higher fraud recall.
- **Model-agnostic API** — the service only relies on predict / predict_proba, so a different winning model needs no API changes.

8.2 Current limitations

- Only 5% of the data is used for training, trading accuracy for speed.
- The 0.5 default threshold is used for the class label; a business-tuned threshold (based on the precision-recall curve) could better balance missed fraud against false alarms.
- CI tests run against the dummy model, so they verify the API contract but not prediction quality.
- Hyper-parameters are fixed; no cross-validation or search is performed.

8.3 Possible extensions

- Train on the full dataset and add gradient-boosted models (e.g. XGBoost, LightGBM).
- Engineer balance-error features such as oldbalanceOrg − amount − newbalanceOrig.
- Tune hyper-parameters and the decision threshold with stratified cross-validation.
- Add a batch prediction endpoint and containerise the API with Docker.
- Register the best model in the MLflow / Azure ML model registry and load it by version.
- Add a CI job that trains on a small fixture dataset and checks a minimum PR-AUC.